CRM Deployment Guide

This document outlines the requirements and step-by-step instructions for deploying the CRM to a live production environment. It covers both Shared Hosting (Manual) and VPS (Command Line) methods.

Prerequisites

Before beginning the installation, ensure the target server meets the following minimum requirements:

- **Web Server:** Apache or Nginx
- **PHP:** Version 8.1 or higher
- **Database:** MySQL 8.0+ or MariaDB 10.3+
- **SSL Certificate:** Mandatory for PWA (Progressive Web App) mobile installation features.
- **Composer:** Installed globally (for VPS)
- **Git:** Installed globally (for VPS)
- **Required PHP Extensions:** BCMath, Ctype, cURL, DOM, Fileinfo, JSON, Mbstring, OpenSSL, PCRE, PDO, Tokenizer, XML, Zip

---

Method 1: Manual Installation (Shared Hosting / cPanel)

This method is ideal for shared hosting environments where command-line access is limited or unavailable.

Step 1: Prepare the Files Locally

1. Open your terminal on your local machine and navigate to your project folder.
2. Run the following command to compile the latest dependencies without development packages:

Bash
```
composer install --optimize-autoloader --no-dev
```

3. Compress your entire project folder into a single **.zip** file. (Exclude the **node_modules** folder and **.git** folder to save space).

Step 2: Upload and Extract

1. Log into your hosting control panel (e.g., cPanel).
2. Open the **File Manager**.
3. For enhanced security, navigate to the directory *above* your public web root (usually above **public_html**).
4. Upload your **.zip** file here and extract it into a folder named crm.

5. Open the extracted **crm/public** folder. Select all files inside (**crucially ensuring that manifest.json**, **sw.js**, **and the icons/ folder are included so the PWA can be installed on mobile devices**), and move them directly into your **public_html** folder.

Step 3: Update Index Paths

1. Inside your **public_html** folder, edit the **index.php** file.
2. Update the paths to point to your secure **crm** directory:

// Change this:
require __DIR__.'/../vendor/autoload.php';
// To this:
require __DIR__.'/../crm/vendor/autoload.php';

// Change this:
$app = require_once __DIR__.'/../bootstrap/app.php';
// To this:
$app = require_once __DIR__.'/../crm/bootstrap/app.php';

Step 4: Database Setup

1. In your hosting panel, navigate to **MySQL Databases**.
2. Create a new database and a new database user.
3. Assign the user to the database with **All Privileges**.
4. Save the Database Name, Username, and Password.

Step 5: Environment Configuration

1. In the File Manager, locate the **.env** file inside your secure **crm** folder (make sure hidden files are visible).
2. Update the following lines to configure your live environment, database, and SMTP mail server:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://yourdomain.com

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=your_new_database_name
DB_USERNAME=your_new_database_user
DB_PASSWORD=your_new_database_password
```

```
MAIL_MAILER=smtp
MAIL_HOST=your_smtp_host
MAIL_PORT=587
MAIL_USERNAME=your_smtp_username
MAIL_PASSWORD="your_password"
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="from_address"
MAIL_FROM_NAME="from_name"
```

Step 6: Finalize Database Tables

1. If your shared hosting provides a Terminal app, open it, navigate to your **crm** folder (**cd crm**), and run:

Bash
```
php artisan migrate –force
php artisan storage:link
```

2. *(Fallback)* If no terminal is available, export your local MySQL database using phpMyAdmin and import the **.sql** file directly into your live database via the hosting panel's phpMyAdmin.

Step 7: Enforce SSL (HTTPS)

1. Navigate to your cPanel's SSL/TLS status page.
2. Run AutoSSL or install a free Let's Encrypt certificate for your domain.
3. *Note: The CRM will not trigger the native "Install Mobile App" prompt for users unless the site is loaded over a verified HTTPS connection.*

Method 2: Command Line Installation (VPS / Dedicated Server)

This method is ideal for VPS environments (e.g., DigitalOcean, AWS, Linode) accessed via SSH tools like PuTTY or the native terminal.

Step 1: Connect and Clone

1. Connect to your server via SSH:

Bash
```
ssh username@your_server_ip
```

2. Navigate to your web directory (e.g., /var/www/):

Bash
```
cd /var/www/
```

3. Clone the repository from GitHub:

Bash
```
git clone https://github.com/yourusername/your-repo-name.git crm
cd crm
```

Step 2: Install Dependencies

1. Install PHP packages required for production:

Bash
```
composer install --optimize-autoloader --no-dev
```

Step 3: Configure the Environment

1. Copy the example environment file to create your active .env:

Bash
```
cp .env.example .env
```

2. Open the .env file using a text editor like Nano:

Bash
```
nano .env
```

3. Update your **APP_ENV** to production, **APP_DEBUG** to false, and fill in your production **DB_*** credentials.

4. Add or update the SMTP settings to ensure the CRM can send emails:

```
MAIL_MAILER=smtp
MAIL_HOST=your_smtp_host
MAIL_PORT=587
MAIL_USERNAME=your_smtp_username
MAIL_PASSWORD="your_password"
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="from_address"
MAIL_FROM_NAME="from_name"
```

5. Save and exit (Ctrl+O, Enter, Ctrl+X).

6. Generate the application's unique encryption key:

Bash
```
php artisan key:generate
```

Step 4: Database and Storage

1. Migrate the database tables:

Bash
```
php artisan migrate --force
```

2. Create the symbolic link for public storage (used for user uploads):

Bash
```
php artisan storage:link
```

Step 5: Set Directory Permissions

1. Ensure the web server (e.g., www-data or nginx) owns the storage and cache directories so the application can write to them:

Bash
```
sudo chown -R www-data:www-data storage bootstrap/cache
sudo chmod -R 775 storage bootstrap/cache
```

Step 6: Web Server Configuration

1. Point your Nginx or Apache virtual host document root to the **public** folder inside your cloned repository (e.g., **/var/www/crm/public**).

2. Restart your web server:

Bash
```
sudo systemctl restart nginx
# OR
sudo systemctl restart apache2
```

System Maintenance & Future Updates

PWA Cache Invalidation : The Progressive Web App utilizes a Service Worker (sw.js) to cache assets locally on the user's device for faster loading. When deploying updates to the CRM (especially CSS, JavaScript, or Blade layout changes), you must invalidate this cache so users receive the latest version.

1. Before pushing your code to the live server, open public/sw.js.

2. Locate the cache definition at the top of the file:

JavaScript
```
const CACHE_NAME = 'studio-crm-cache-v1';
```

3. Increment the version number (e.g., change **v1** to **v2**).

4. When the updated **sw.js** file reaches the live server, users' mobile devices will automatically detect the version change, clear their old cached files, and download the latest application code.